

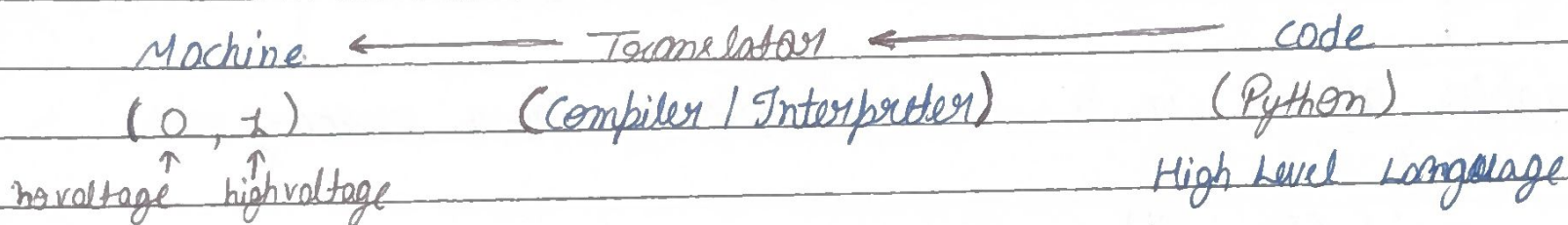
Python Programming

Chapter 1

Variables and Data types

Introduction to Python

Programming :



What is Python?

- Python is simple & easy.
- Free & Open source.
- High Level Language.
- Developed by Guido van Rossum.
- Portable

Our First Program

```
print("Hello World")
```

Output → Hello World

Python Character Set :

- Letters - A to Z, a to z
- Digits - 0 to 9
- Special Symbols - + - * / etc.
- White spaces - Blank Space, tab, carriage return, newline, formatted
- Other characters - Python can process all ASCII and Unicode characters as part of data or literals

Variables:

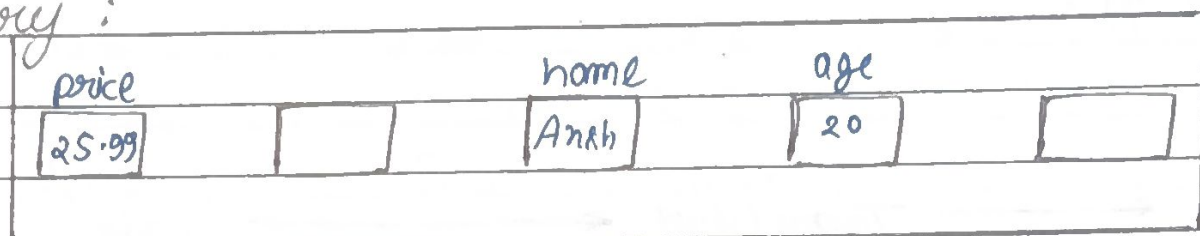
A variable is a name given to a memory location in a program.

ex- name = "Ansh"

age = 20

price = 25.99

Memory:



variables allocate in a random place in a memory

example -:

print("name")

output → name

If we want to print data which store in a variable then we don't need the "" double colon signs.

print(name) // far above

output → Ansh

Rules for Identifiers:

1. Identifiers can be combination of uppercase and lowercase letters, digits or an underscore (_).

So myVariable, variable_1, variable_for_print all are valid python identifiers.

2. An Identifier can not start with digit. So while variable_1 is valid, 1variable is not valid.

3. We can't use special symbols like !, #, @, %, \$ etc in our Identifier.

4. Identifier can be of any length.

Data Types

- Integer $\rightarrow$ +ve, -ve, 0 ex (25, -25, 0)
- String $\rightarrow$ "Ansh", 'Ansh', ""Ansh""
- Float $\rightarrow$ 3.99, 2.5, 9.0
- Boolean $\rightarrow$ True, False
- None $\rightarrow$ ex - a = None (not stored any value in a)

Keywords

Keywords are reserved words in python.

and	else	in	return
as	except	is	True
assert	finally	lambda	try
break	False	nonlocal	with
class	for	None	while
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

Python is a case sensitive language

If we write

Apple apple

Both are different variables

Types of Tokens

Punctuators:

Punctuators are symbols to organize sentence structure in programming.

(), { }, @, [], # etc.
 -=, +=, /=, *=, //=, = etc.



Python is Typed Language

Implicit

ex - python

ex - age = 23

Explicit

ex - Java, C++, C etc.

ex - int age = 23

* means in python there is no need to define the type of variables.

Expression Execution

- String & Numeric values can operate together with *

ex - A, B = 2, 3

Txt = "@" ← string

print (2 * Txt * 3)

output → @ @ @ @ @ @

A = 2
B = 3

↑
repeat

- String & String can operate with +

ex - A, B = "2", 3

Txt = "@"

print ((A + Txt) * B)

output → 2@ 2@ 2@

↑
concatenation

2 → number (int)

"2" → string

- Numeric values can operate with all arithmetic operators

ex- $A, B = 2, 3$

$C = 4$

`print (A+B*C)`

output $\rightarrow 14$

- Arithmetic expression with Integer and float will result in float.

ex- $A, B = 10, 5.0$

$C = A * B$

`print (C)`

output $\rightarrow 50.0$ (✓) not 50 (X)

- Result of division operator with two integers will be float.

ex- $A, B = 1, 2$

$C = A / B$

`print (C)`

output $\rightarrow 0.5$

- Integer division with float and int will give int displayed as float.

ex- $A, B = 1.5, 3$

$C = A // B$

`print (C, A/B)`

output $\rightarrow 0.0, 0.5$

- `floor` gives closest integer, which is lesser than or equal to the float value.

Result of $(A // B)$ is same as (A/B)

ex - 1) $A, B = 12, 5$

$$C = A // B$$

print(C)

output $\rightarrow 2$

2) $A, B = -12, 5$

$$C = A // B$$

print(C)

output $\rightarrow -3$

3) $A, B = 12, -5$

$$C = A // B$$

print(C)

output $\rightarrow -3$

• Remainder is negative when denominator is negative

ex - 1) $A, B = -5, 2$

$$C = A \% B$$

print(C)

output $\rightarrow 1$

2) $A, B = 5, 2$

$$C = A \% B$$

print(C)

output $\rightarrow 1$

3) $A, B = 5, -2$

$$C = A \% B$$

print(C)

output $\rightarrow -1$

<u>Numerator</u>	$\frac{+}{+}$	$\frac{-}{-}$	$\frac{+}{-}$	$\frac{-}{+}$
<u>Denominator</u>	$\frac{+}{+}$	$\frac{-}{-}$	$\frac{+}{-}$	$\frac{-}{+}$
remainder $\Rightarrow$	+ve	+ve	-ve	+ve

completed $\rightarrow$ 1 hour 5 minutes of 20 hours 25 minutes

Comments in Python

single line comment

""" This is a multi-line comment """

print("Hello") ← won't be printed

print("World") ← will be printed

Input in Python

input() statement is used to accept values (using keyboard) from user

- string input

name = input("name : ")

- int input

age = int(input("age : "))

- float input

price = float(input("price : "))

Types of Operators

An operator is a symbol that performs a certain operation betⁿ operands

- Arithmetic Operators

(+ , - , * , / , // , % , **)

- Relational / Comparison Operators

(== , != , > , < , >= , <=)

- Assignment Operators

(= , -= , *= , /= , %= , //= , **=)

- Logical Operators

(not , and , or)

- Membership Operators

(in , not in)

- Identity Operators

(is , is not)

- Bitwise Operators

(& , | , ^)
AND OR XOR

Conditional Statements

if - elif - else (Syntax)

if (condition):

Statement 1

elif (condition):

Statement 2

else:

Statement N

after this (:) ↓

4 space gap is permanent

because python is indentation

language

proper spacing

Single Line If / Ternary Operator:

① $\langle \text{var} \rangle = \langle \text{val}_1 \rangle \text{ if } \langle \text{condition} \rangle \text{ else } \langle \text{val}_2 \rangle$

ex- `food = input("Food: ")`
`eat = "Yes" if food == "cake" else "no"`
`print(eat)`

② $\langle \text{stt}_1 \rangle \text{ if } \langle \text{condition} \rangle \text{ else } \langle \text{stt}_2 \rangle$

ex- `food = input("Food: ")`
`print("sweet") if food == "cake" or food == "jalebi" else`
`print("not sweet")`

③ Clever If / Ternary Operator

$\langle \text{var} \rangle = (\text{false_val}, \text{true_val}) [\langle \text{condition} \rangle]$

ex- `age = int(input("age: "))`
`vote = ("yes", "no") [age < 18]`
 ex- `age = 17`
 then condⁿ true an execute true-val
 output $\rightarrow$ no

Best Practices:

simple instructions

one instruction per task

short & meaningful variable names

use appropriate comments

avoid complex expressions

Comments in Python

`# Single line comment`

`""" Multi Line
Comment
"""`

2 hours 2 minutes 0 seconds completed

Type Conversion:

Type conversion automatic done by the interpreter

ex-① `a, b = 1, 2.5`

`sum = a + b` `# a = 1.0, b = 2.5 = 1.0 + 2.5 = 3.5`
`print (sum: ", sum)`

output $\rightarrow$ `sum: 3.5`

② `a = "2"`

`b = 1.5`

`sum = a + b`

`print (sum)`

output $\rightarrow$ error string not add in float value
so for this example we use type casting.

Type Casting:

Type casting is explicit work done by the user.

ex- `a = "2"`

`b = 1.5`

`c = int(a)`

`sum = b + c`

output $\rightarrow$ `3.5`

Input in Python:

`input()` statement is used to accept values (using keyboard) from user.

`input()` # result for `input()` is always a string
So we need type casting for getting values in `int`, `float` etc.

`int(input())` # `int`

`float(input())` # `float`

Chapter : 2

Strings & Conditional Statements

Strings: String is data type that stores a sequence of characters.

escape sequence characters:

`\n` : → next line

`\t` : → tab space

`str1 = "My name \n Ansh"`

`print(str1)`

output → My name
Ansh

Basic Operations on String:

- Concatenation (+)

ex- `str1 = "Ansh"`

`str2 = "Shukla"`

`print(str1 + str2)`

output → Ansh Shukla

72

• length of str $\rightarrow$ `len(str)`

ex- `str1 = "Ansh"`

`len1 = len(str1)`

`print(len1)`

output $\rightarrow$ 4

Indexing : index $\rightarrow$ position

Apna - College

0 1 2 3 4 5 6 7 8 9 10 11

`str1 = "Apna - College"`

`str1[0]` is A `str1[1]` is p

`str1[0] = B` # we can not change the value by indexing

Slicing :

Accessing parts of a string.

`str[starting_idx : ending_idx]` # ending idx is not included

ex-

`str = "ApnaCollege"`

`str[1:4]` is "pna"

`str[:4]` is "Apna" same as `str[0:4]`

`str[1:]` is "pnaCollege" same as `str[1:len(str)]`

`str[:]` is "ApnaCollege"

Negative Index

Apple

-5 -4 -3 -2 -1

`str = "Apple"`

`str[-3:-1]` is "pl"

2 hours 52 minutes 26 seconds completed

String Functions

str = "I am a coder."

- str.endswith("er.") # returns true if string ends with substring
- str.capitalize() # capitalizes 1st char
- str.replace(old, new) # replaces all occurrences of old with new.
value value
- str.find(word) # returns 1st index of 1st occurrence
- str.count("am") # counts the occurrence of substring in string

3 hours 12 minutes 91 seconds

Conditional Statements

Date - 6/02/26

if - elif - else (Syntax)

if (condition):

Statement 1

elif (condition):

Statement 2

else:

Statement N

↑ ↑ ↑ ↑

4-space
Indentation is used
in Python for define
the block

3 : 27 : 17 Completed

Chapter 3

List and Tuples

List in Python: Lists are mutable (can change)

A built in data type that stores set of values.

It can store elements of different types (integer, float, string, etc.)

Example:

```
marks = [87, 69, 52, 50] # marks[0], marks[1] ... etc.
student = ["Karan", 85, "Delhi"] # student[0], student[1] ... etc.
student[0] = "Arijun" # allowed in python
len(student) # returns length
```

List Slicing: Similar to string slicing

list_name [starting_idx : ending_idx] # ending_idx is not included.

-5	-4	-3	-2	-1
0	1	2	3	4

Example: marks = [87, 69, 33, 95, 76]

marks[1:4] is [69, 33, 95]

marks[:4] is same as marks[0:4]

marks[1:] is same as marks[1:len(marks)]

marks[-3:-1] is [33, 95]

List Methods (Functions):

list = [2, 1, 3]

list.append(4) # adds one element at the end [2, 1, 3, 4]

list.sort() # sorts in ascending order [1, 2, 3]

list.sort(reverse = True) # sorts in descending order [3, 2, 1]

list.reverse() # reverse list [3, 1, 2]

list.insert(idx, el) # insert element at index

list.remove(1) # removes first occurrence of element [2, 3]

list.pop(idx) # removes element at idx

Tuples in Python:

data unchanged
↑

A built in data type that lets us create immutable sequences of values.

```
tup = (87, 89, 33, 95, 76) # tup[0], tup[1] - -
```

```
tup[0] = 93 # NOT allowed in python
```

```
tup1 = ()
```

```
tup2 = (1,) # we write single values with comma because without
```

```
tup3 = (1, 2, 3) it can take as tuple it take a integer like
```

example:

```
① tup = (2,)
```

```
print(type(tup))
```

```
<class 'tuple'>
```

```
② tup = (2)
```

```
print(type(tup))
```

```
<class 'int'>
```

tuple slicing: same as list slicing

Tuple Methods:

```
tup = (2, 1, 3, 1)
```

```
① tup.index(el) # return index of first occurrence
```

```
tup.index(1) is 1
```

```
② tup.count(el) # counts total occurrences
```

```
tup.count(1) is 2
```


Chapter : 4

Dictionary in Python :

Dictionaries are used to store data values in Key: value pairs. They are unordered, mutable (changeable) & don't allow duplicate keys.

eg: dict = {
 "name" : "Arth",
 "cgpa" : "9.6",
 "marks" : [98, 97, 95],
 "key" : value

dict["name"]; dict["c_gpa"]; dict["marks"]
dict["key"] = "value" # to assign or add new

Nested dictionary : make a dictionary and then make a dictionary inside the dictionary.

example :

Student = \$

"home" : "Anesh Shukla"

"Subjects" : \$

 "phy" : 97,

 "chem" : 98,

 "math" : 95

```
# nested dictionary
print(student["subjects"]["chem"])
```

use two square brackets ([]) to access the subject.

Dictionary Methods :

- myDict.keys() # returns all keys.
- myDict.values() # returns all values.
- myDict.items() # returns all (key, val) pairs as tuples.
- myDict.get("key") # returns the key according to value.
- myDict.update(newDict) # inserts the specified items to the dictionary

Duplicate keys are not allowed in Dictionaries.

Set in Python :

no index access like list

- Set is the collection of the unordered items.
- Each element in the set must be unique & immutable but set is mutable because we can add and remove the values.
- nums = {1, 2, 3, 4}
- set2 = {1, 2, 2, 2}
- # repeated elements stored only once, so it resolved to {1, 2}

null_set = set() # empty set syntax

Set Methods

- set.add(el) # adds an element
- set.remove(el) # removes the element
- set.clear() # empties the set
- set.pop() # removes a random value
- set.union(set2) # combines both set values & returns new set
- set.intersection(set2) # combines common values & returns new

Chapter : 5

Loops

Loops in Python:

Loops are used to repeat instructions.

while loops

while condition:

 # some work

→ while

→ for

example: write a python program which print numbers 1 to 5.

code:-

```
i = 5
while i >= 1:
    print(i)
    i -= 1
```

```
print("Loop ended")
```

Output:-

5
4
3
2
1

Loop ended.

Break & Continue:

- Break used to terminate the loop when encountered.
- Continue terminates execution in the current iteration & continues execution of the loop with the next iteration.

for loops:- Loops are used for sequential traversal.
For traversing list, string, tuples etc.

Syntax:-

we can write any keyword like i, j etc.

```
for el in list:
    # some work
```

ex- list = [1, 2, 3]
for el in list:
 print(el)

for loop with else:-

```
for el in list:
    # some work
else:
    # work when loop ends
```

ex- for el in list:
 print(el)
else:
 print("END")

range function:-

range() :- Range function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax:- range(start?, step, stop?)

ex- for el in range(5):
 print(el)

ex- range(1, 5)

ex- range(1, 5, 2)

pass Statement:- pass is a null statement that does nothing. It is used as a placeholder for future code.

ex-

```
for el in range(10):
    pass
```


Functions in Python :-

Block of statements that perform a specific task.

```
def func_name (para1, param2 ---):
```

Function Definition

```
    # some work
    return val
```

```
func_name (arg1, arg2 ---) # function call
```

```
ex- def sum(a, b):
      s = a + b
      return s
```

```
print (sum (2, 3))
```

Types of function :-

↓

Built-in Functions

```
ex- print()
      len()
      type()
      range()
```

↓

User defined Functions

Default Parameters :- Assigning a default value to parameter, which is used when no argument is passed.

Recursion:

When a function calls itself repeatedly.

ex-

$\swarrow$ recursive function
`def show(n):`
 `if (n == 0):` $\swarrow$ Base case

`return`

`print(n)`

`show(n-1)`

`show(5)`

Output:

5

4

3

2

1

Q-1) Write a recursive function to calculate the sum of first n natural numbers.

`def sum(n):`

`if (n == 0):`

`return 0`

`return n + sum(n-1)`

`print("Sum:", sum(4))`

Output: Sum : 10

Q-2) Write a recursive function to print all elements in a list.

`list = [1, 2, 3, 4, 5]`

`def print_from_list(list):`

`for item in list:`

`print(item)`

`print_from_list(list)`

Date: 7/07/26

Chapter: 7 File I/O

• File Input/Output in Python :-

Python can be used to perform operations on a file.
(read and write data)

Types of all files :-

1) Text Files : .txt , .docx , .log etc.

2) Binary Files : .mp4 , .mov , .png , .jpeg etc.

• Open, read & close File :-

We have to open a file before reading or writing.

```
f = open("file_name", "mode")
```

sample.txt
demo.docx

r: read mode
w: write mode

```
data = f.read()  
f.close()
```

After opening the file we have to close file after the work done on those file because if we left file open then there is a risk to data leakage or data breaching.

Character

Meaning

'H'

open for reading (default).

'w'

open for writing, truncating the file first.

(X)

create a new file and open it for writing.

'a'

open for writing, appending to the end of the file if it exists.

'b'

binary mode

14

text mode (default)

(+)

open a disk file for updating (reading and writing)

Reading a file :-

- `data = f.read()` # reads entire file
- `data = f.readlines()` # reads one line at a time
- `data = f.read(5)`
↑

We can some specific characters of data

Writing to a file :-

```
f = open("demo.txt", "w")  
f.write("This is a new line") # overwrites the entire file
```

```
f = open("demo.txt", "a")  
f.write("this is a new line") # adds to the file
```


	↓	↓	↓	↓
r^+ →	read + overwrite	(pointer at the beginning)	- no-truncate	
w^+ →	read + overwrite		- truncate	
a^+ →	read + append	(pointer at the end)	- no-truncate	

- With Syntax :-

with open ("demo.txt", "a") as f:

data = f.read()

- Deleting a File :-

using the os module

Module (like a code library) is a file written by another programmer that generally has a functions we can use.

import os

os.remove(filename)

Objed = instance

25

Date :- 10/07/25

Chapter 8

OOPS

(Object-Oriented Programming System).

OOP in Python:

To map with real world scenarios, we started using objects in code.

This is called object oriented programming.

- Class & Object in Python:

Class is a blueprint for creating objects.

creating class

```
class Student:
```

```
    name = "Karan Kumar"
```

creating object (instance)

```
s1 = Student()
```

```
print(s1.name)
```

- --init-- Function :-

Construction

All classes have a function called `--init--()`, which is always executed when the class is being initiated.

creating class

```
class Student:
```

```
    def __init__(self, fullname):
```

```
        self.name = fullname
```

creating object

```
s1 = Student("Karan")
```

```
print(s1.name)
```


The self parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

- Class & Instance Attributes:

Class.attr

obj.attr

- Methods :-

Methods are functions that belong to objects.

creating class

creating object

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def hello(self):
```

```
        print("hello", self.name)
```

```
s1 = Student("Karon")
```

```
s1.hello()
```

- Static Methods:

Methods that don't use the self parameter (work at class level)

```
class Student:
```

```
    @staticmethod # decorator
```

```
    def college():
```

```
        print("ABC College")
```

Decorators allow us to wrap another function in order to extend the behaviour of the wrapped function, without permanently modifying it.

- Important :

- Abstraction :-

Hiding the implementation details of a class and only showing the essential features to the user.

→ means showing only necessary details.

- Encapsulation :

Wrapping data and functions into a single unit (object).

Date - 12-07-26

Chapter - 8

Part - 2

OOPS :-

(Object Oriented Programming System)

- del Keyword :-

Used to delete object properties or object itself.

ex - `del s1.name`
`del s1`

- Private (like) attributes & methods :

→ Conceptual Implementations in Python :

Private attributes & methods are meant to be used only within the class and are not accessible from outside the class.

- Inheritance :-

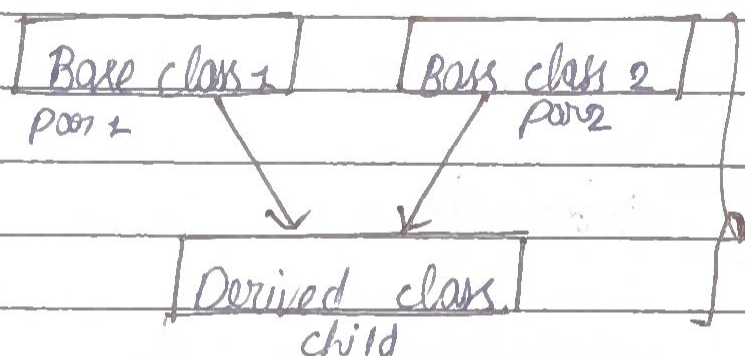
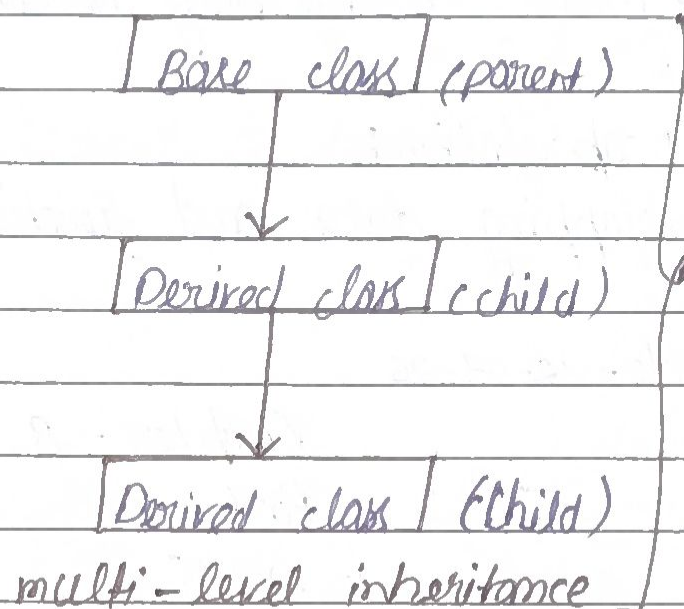
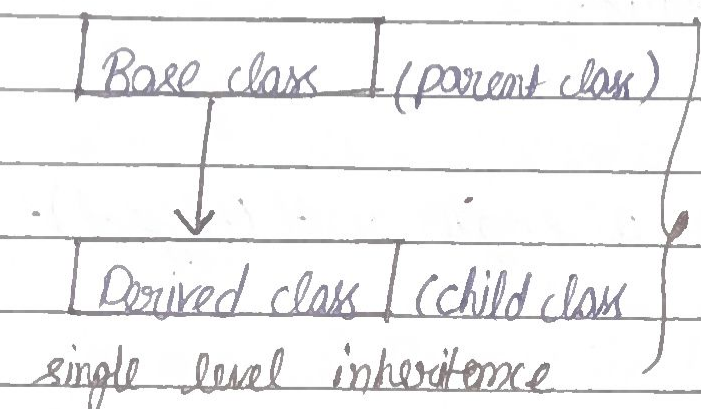
When one class (child/derived) derives the properties & methods of another class (parent/base).

ex - `class Car :`

`class ToyotaCar(Car):`

• Types of Inheritance :

- Single Inheritance
- Multi-level Inheritance
- Multiple Inheritance



Multiple Inheritance

• Super method :-

super() method is used to access methods of the parent class.

• class method :-

A class method is bound to the class & receives the class as an implicit first argument.

Note - static method can't access or modify class state & generally for utility.

class Student:

@classmethod # decorator

def college(cls):

pass

• Property Decorator

We use @property decorator on any method in the class to use the method as a property.

Date - 16-07-26

• Polymorphism : Operator Overloading :

When the same operator is allowed to have different meaning according to the context.

Operators & Dunder functions :

$a+b$ # addition $a.__add__(b)$

$a-b$ # subtraction $a.__sub__(b)$

$a*b$ # multiplication $a.__mul__(b)$

a/b # division $a.__truediv__(b)$

$a\%b$ # modulo $a.__mod__(b)$

Python Completed Now do a Mini Project in Python.

Mini Project

Guess the Number

code :- import random

target = random.randint(1, 1000)

while True:

user_input = input("Guess the no. or Quit(Q): ")

if (user_input == "Q"):

break

if (user_input == target):

print("Successfully guessed the number")

break

else:

if (user_input > target):

print("Your no. is too greater guess the smaller number")

else:

print("Your no. is too smaller guess the greater number")

print("----- Game Over -----")